

Python

Manuale di preparazione e ripasso di inizio anno

A. Adinolfi
Dipartimento di Informatica
Liceo Scientifico Statale “Nino Cortese” – Maddaloni (CE)

Abstract—Questo manuale raccoglie, in forma sintetica, le nozioni fondamentali di programmazione in Python necessarie per affrontare la scheda di ripetizione di inizio anno. Sono richiamati i concetti di variabile, tipo di dato, struttura di controllo, ciclo, lista, funzione e i principi essenziali di progettazione di un algoritmo, insieme a cenni sul modulo `random` e sull'efficienza degli algoritmi. Il documento non sostituisce il libro di testo, ma ne costituisce un compendio operativo da consultare prima e durante lo svolgimento degli esercizi.

Index Terms—Python, variabili, cicli, liste, funzioni, algoritmi, efficienza, `random`

I. INTRODUZIONE

Questo manuale è organizzato per argomenti, in ordine di difficoltà crescente, in modo coerente con la struttura della scheda di ripetizione. Per ciascun argomento vengono richiamate la sintassi essenziale e le regole più facilmente dimenticate durante la pausa estiva. Gli esempi di codice sono volutamente generici: non risolvono gli esercizi della scheda, ma ne illustrano i costrutti necessari.

II. VARIABILI E TIPI DI DATO

Una variabile è un nome simbolico associato a un valore memorizzato in memoria centrale. In Python una variabile non richiede una dichiarazione esplicita del tipo: il tipo è associato al valore, non al nome, e può cambiare durante l'esecuzione (*tipizzazione dinamica*).

```
1 eta = 16 # int
2 altezza = 1.72 # float
3 nome = "Marco" # str
4 maggiorenne = False # bool
```

I tipi di dato fondamentali sono riassunti in Tabella I.

TABLE I
TIPI DI DATO FONDAMENTALI

Tipo	Descrizione
<code>int</code>	numero intero (es. 7, -3)
<code>float</code>	numero decimale (es. 3.14)
<code>str</code>	sequenza di caratteri (stringa)
<code>bool</code>	valore logico <code>True/False</code>

La funzione `type(x)` restituisce il tipo del valore contenuto in `x` ed è utile per verificare, in fase di debug, se una conversione è avvenuta correttamente.

III. INPUT, OUTPUT E CONVERSIONE DEI DATI

La funzione `input()` acquisisce sempre una stringa da tastiera, anche se l'utente digita un numero: per questo motivo, quando il dato deve essere trattato come numero, occorre convertirlo esplicitamente con `int()` oppure `float()`.

```
1 testo = input("Inserisci un numero: ")
2 numero = int(testo) # oppure float(testo)
3 print("Il doppio è:", numero * 2)
```

La funzione `print()` accetta più argomenti separati da virgola, stampati automaticamente separati da uno spazio; l'argomento `sep` permette di cambiare il separatore e `end` il carattere finale (di default `"\n"`).

TABLE II
FUNZIONI DI CONVERSIONE PIÙ COMUNI

Funzione	Effetto
<code>int(x)</code>	converte <code>x</code> in numero intero
<code>float(x)</code>	converte <code>x</code> in numero decimale
<code>str(x)</code>	converte <code>x</code> in stringa

Errore tipico: tentare operazioni aritmetiche su un dato non convertito genera un `TypeError`, perché Python non converte automaticamente una stringa in numero.

IV. OPERATORI

Gli operatori aritmetici (Tabella III) seguono le regole di precedenza della matematica; le parentesi tonde possono sempre essere usate per rendere esplicito l'ordine di valutazione.

TABLE III
OPERATORI ARITMETICI

Operatore	Significato
<code>+</code> <code>-</code> <code>*</code> <code>/</code>	somma, differenza, prodotto, divisione (float)
<code>//</code>	divisione intera (quoziente)
<code>%</code>	resto della divisione intera (modulo)
<code>**</code>	elevamento a potenza

Gli operatori relazionali (`==`, `!=`, `<`, `>`, `<=`, `>=`) restituiscono sempre un valore booleano e si combinano con gli operatori logici `and`, `or` e `not` per costruire condizioni composte.

```
1 n = 15
2 pari = (n % 2 == 0)
3 multiplo = (n % 3 == 0) and (n % 5 == 0)
```

Attenzione: l'uguaglianza si verifica con `==` (doppio simbolo); un solo `=` è l'operatore di assegnazione e produce un errore di sintassi se usato in una condizione.

V. STRUTTURE CONDIZIONALI

La struttura `if / elif / else` permette di eseguire blocchi di istruzioni alternativi in base al valore di una o più condizioni. In Python l'appartenenza di un'istruzione a un blocco è determinata dall'indentazione (di norma quattro spazi), non da parentesi graffe.

```
1 if n % 15 == 0:
2     print("Multiplo di 3 e di 5")
3 elif n % 3 == 0:
4     print("Multiplo di 3")
5 elif n % 5 == 0:
6     print("Multiplo di 5")
7 else:
8     print("Nessuno dei due")
```

Le condizioni possono essere annidate (un `if` dentro un altro `if`) quando la logica del problema lo richiede, ad esempio per verificare prima la validità di un dato e poi classificarlo.

VI. CICLI E CICLI ANNIDATI

Il ciclo `while` ripete un blocco finché una condizione resta vera; è indicato quando non si conosce a priori il numero di ripetizioni (per esempio nella validazione dell'input). Il ciclo `for` scorre gli elementi di una sequenza (una lista, una stringa, o l'intervallo prodotto da `range()`) ed è indicato quando il numero di ripetizioni è noto o legato alla lunghezza di una collezione di dati.

```
1 i = 0
2 while i < 5:
3     print(i)
4     i += 1
5
6 for i in range(5):
7     print(i)
```

I *cicli annidati* (un ciclo dentro un altro) sono necessari quando occorre combinare due grandezze indipendenti, per esempio riga e colonna di una tabella, oppure ogni coppia di elementi di due liste.

```
1 for riga in range(3):
2     for colonna in range(3):
3         print(riga, colonna)
```

Errore tipico: dimenticare di aggiornare la variabile di controllo in un ciclo `while` produce un ciclo infinito.

VII. CONTATORI, ACCUMULATORI E VALIDAZIONE DELL'INPUT

Un *contatore* è una variabile inizializzata (di solito a 0) e incrementata di un valore fisso a ogni iterazione, usata per contare quante volte si verifica un evento. Un *accumulatore* è una variabile che raccoglie progressivamente un risultato (una somma, un prodotto, il massimo trovato finora).

```
1 somma = 0 # accumulatore
2 conteggio_pari = 0 # contatore
3 for x in [3, 8, 5, 12, 7]:
4     somma += x
5     if x % 2 == 0:
6         conteggio_pari += 1
```

La *validazione dell'input* consiste nel ripetere la richiesta di un dato finché l'utente non inserisce un valore accettabile, tipicamente all'interno di un ciclo `while`:

```
1 voto = int(input("Voto (1-10): "))
2 while voto < 1 or voto > 10:
3     print("Valore non valido, riprova.")
4     voto = int(input("Voto (1-10): "))
```

VIII. LISTE E SEQUENZE

Una lista è una collezione ordinata e modificabile di valori, anche di tipo diverso, racchiusa fra parentesi quadre. Gli elementi si indicizzano a partire da 0; l'indice `-1` indica l'ultimo elemento.

```
1 voti = [7, 5, 9, 6, 8]
2 voti.append(10) # aggiunge un elemento in coda
3 primo = voti[0]
4 ultimo = voti[-1]
5 n = len(voti) # numero di elementi
```

TABLE IV
OPERAZIONI COMUNI SULLE LISTE

Operazione	Effetto
<code>len(lista)</code>	numero di elementi
<code>lista.append(x)</code>	aggiunge <code>x</code> in coda
<code>lista[i]</code>	accede all'elemento di indice <code>i</code>
<code>for x in lista:</code>	scorre tutti gli elementi

Molti esercizi richiedono di *non* utilizzare funzioni predefinite come `sum()`, `max()` o `min()`, proprio per allenare la costruzione autonoma di accumulatori tramite un ciclo `for`.

IX. FUNZIONI

Una funzione è un blocco di codice riutilizzabile, identificato da un nome, che può ricevere parametri e restituire un risultato con `return`. Le funzioni permettono di scomporre un problema complesso in sotto-problemi più semplici.

```
1 def e_pari(numero):
2     return numero % 2 == 0
3
4 def statistiche(lista):
5     totale = 0
6     for x in lista:
7         totale += x
8     media = totale / len(lista)
9     return totale, media
10
11 tot, media = statistiche([4, 8, 6])
```

Una funzione può restituire più valori separati da virgola: in fase di chiamata vengono assegnati, nello stesso ordine, a più variabili contemporaneamente.

X. ALGORITMI, PROBLEM SOLVING ED EFFICIENZA

Un algoritmo è una sequenza finita e non ambigua di passi che risolve un problema. Nella progettazione di un algoritmo è utile individuare: i dati in ingresso, il risultato atteso, le variabili di stato necessarie durante l'elaborazione (per esempio un massimo parziale o una sequenza corrente).

L'*efficienza* di un algoritmo si valuta in base al numero di operazioni necessarie al crescere della dimensione del problema.

Un esempio classico è la verifica di primalità di un numero n : controllare tutti i divisori da 2 a $n - 1$ richiede molte più operazioni rispetto a controllare i soli divisori fino a $\sqrt{n}$, poiché se n ha un divisore maggiore di $\sqrt{n}$ deve necessariamente averne anche uno minore.

```
1 def primo_base(n):
2     for d in range(2, n):
3         if n % d == 0:
4             return False
5     return True
6
7 def primo_ottimizzato(n):
8     d = 2
9     while d * d <= n:
10        if n % d == 0:
11            return False
12        d += 1
13    return True
```

XI. IL MODULO `RANDOM` E LE SIMULAZIONI

Il modulo `random` consente di generare valori casuali, utili per simulare fenomeni non deterministici (lancio di un dado, estrazione di una carta). Va importato all'inizio del programma con `import random`.

```
1 import random
2
3 dado = random.randint(1, 6)    # intero tra 1 e 6, estremi
                                # inclusi
4 scelta = random.choice(["A", "B", "C"])
```

Nelle simulazioni ripetute (per esempio molti lanci di un dado), è utile tenere un contatore per ogni possibile risultato, aggiornato a ogni iterazione, per calcolare infine le frequenze assolute e relative.

XII. SINTESI E CONSIGLI PER LO SVOLGIMENTO

Prima di scrivere codice, conviene sempre: (i) individuare i dati in ingresso e il risultato atteso; (ii) scegliere le variabili necessarie (contatori, accumulatori, liste); (iii) scrivere lo schema dell'algoritmo, anche a parole, prima di tradurlo in Python; (iv) verificare il programma con dati di prova semplici, di cui si conosce già il risultato atteso. Errori di indentazione, dimenticanza della conversione dei dati e cicli `while` privi di aggiornamento della condizione sono le cause più frequenti di malfunzionamento nei programmi degli studenti.